Django Complete Study Notes — Module 18

1. Introduction to Django

a) Overview of Django

Django is a high-level Python web framework used to build secure, maintainable, and database-driven web applications quickly.

It follows the **MVT (Model–View–Template)** architectural pattern.

Examples of applications you can build with Django:

- Student management systems
- E-commerce websites
- Blog platforms
- REST APIs and backend services
- Authentication and dashboard applications

b) Key features and benefits

- **ORM:** Interact with databases using Python.
- **Built-in admin panel:** Manage database records.
- **URL routing:** Map URLs to views.
- **Template engine:** Generate dynamic HTML pages.
- **Security:** Built-in CSRF protection and other security features.
- **Authentication:** User login, permissions and authentication tools.
- **Migrations:** Manage database schema changes.
- **Reusable apps:** Divide a project into separate functional components.
- **Rapid development:** Many common features are already provided.

c) Django project structure and components

A Django project can contain multiple apps.

mysite/

├── manage.py

├── mysite/

```
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
└── students/
    ├── migrations/
    ├── templates/
    ├── models.py
    ├── views.py
    ├── urls.py
    ├── admin.py
    ├── apps.py
    └── tests.py
```

File	Purpose
`manage.py`	Runs Django management commands.
`settings.py`	Configures apps, databases, middleware and templates.
`urls.py`	Defines URL routing.
`wsgi.py`	WSGI application entry point.

`asgi.py`	ASGI application entry point.
`models.py`	Defines database models.
`views.py`	Handles requests and returns responses.
`admin.py`	Registers models with Django admin.
`apps.py`	Contains app configuration.
`tests.py`	Contains automated tests.
`migrations/`	Stores database schema migration files.

Project vs app

- Project: The overall website and its configuration.
- App: A component responsible for a particular feature, such as students or courses.

---

2. Setting Up a Django Project

a) Installing Django

Create and activate a virtual environment if required.

python -m venv venv

Windows Command Prompt:

venv\Scripts\activate.bat

Windows PowerShell:

.\venv\Scripts\Activate.ps1

Install Django:

python -m pip install django

Check the installed version:

python -m django --version

b) Creating a new project

django-admin startproject mysite

cd mysite

Create an app:

python manage.py startapp students

Add the app to `settings.py`:

```
INSTALLED_APPS = [
    # Existing Django apps...
    'students',
]
```

c) Understanding the configuration files

`settings.py`

Contains the project's configuration.

```
DEBUG = True


ALLOWED_HOSTS = []


INSTALLED_APPS = [
```

Django applications

]

- `DEBUG = True`: Enables detailed debugging information during development.
- `ALLOWED_HOSTS`: Specifies permitted hostnames.
- `INSTALLED_APPS`: Lists enabled Django applications.
- `MIDDLEWARE`: Defines the middleware stack.
- `DATABASES`: Configures database connections.
- `TEMPLATES`: Configures template rendering.

Use `DEBUG = False` and configure `ALLOWED_HOSTS` appropriately in production.

`urls.py`

Maps incoming URL paths to views.

from django.urls import path

urlpatterns = []

`wsgi.py` and `asgi.py`

These files expose application entry points for compatible application servers.

d) Running the development server

python manage.py runserver

By default, Django serves the development site at:

http://127.0.0.1:8000/

The development server is intended for development, not production deployment.

Other useful commands

python manage.py check

python manage.py shell

python manage.py createsuperuser

python manage.py test

---

3. Django Models

a) Introduction to Django ORM

ORM stands for **Object-Relational Mapper**.

It allows you to interact with database records using Python objects and methods instead of writing SQL for every operation.

SQL:

SELECT * FROM students;

Django ORM:

Student.objects.all()

b) Creating and defining models

In `students/models.py`:

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    marks = models.IntegerField()


    def __str__(self):
        return self.name
```

Each model generally represents a database table, and each model field describes a database column.

c) Fields and field types

Field	Purpose
`CharField`	Short text
`TextField`	Longer text
`IntegerField`	Integer values
`FloatField`	Floating-point numbers
`DecimalField`	Decimal numbers with configured precision
`BooleanField`	True or false
`DateField`	Date
`DateTimeField`	Date and time
`EmailField`	Email address
`ForeignKey`	Many-to-one relationship
`OneToOneField`	One-to-one relationship

`ManyToManyField`	Many-to-many relationship
`FileField`	Uploaded files
`ImageField`	Uploaded images

Example:

```
name = models.CharField(max_length=100)
age = models.IntegerField(default=18)
is_active = models.BooleanField(default=True)
```

d) Model methods

You can define custom methods inside models to implement behaviour related to the data.

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    marks = models.IntegerField()

    def has_passed(self):
        return self.marks >= 40

    def __str__(self):
        return self.name
```

Usage:

```
student = Student.objects.get(id=1)
```

student.has_passed()

`__str__()` returns a human-readable string representation of the object. It must return a string, not a list or number.

e) Making and applying migrations

After creating or modifying models, run:

python manage.py makemigrations

python manage.py migrate

- `makemigrations`: Creates migration files describing model changes.
- `migrate`: Applies pending migrations to the database.

Basic ORM operations

Create:

Student.objects.create(

    name="Ali",

    age=18,

    marks=90

)

Read:

Student.objects.all()

Student.objects.get(id=1)

Filter:

Student.objects.filter(marks__gt=80)

Update:

student = Student.objects.get(id=1)

student.marks = 95

student.save()

Delete:

student = Student.objects.get(id=1)

student.delete()

Common QuerySet methods

Method	Purpose
`all()`	Retrieve all matching records
`filter()`	Retrieve matching records
`exclude()`	Exclude matching records
`get()`	Retrieve one matching object
`order_by()`	Sort records
`first()`	Return the first object or `None`
`last()`	Return the last object or `None`
`count()`	Count records
`exists()`	Check whether matching records exist

`aggregate()`	Calculate an overall summary
`annotate()`	Add calculated values to individual objects or groups

Filtering operators

Student.objects.filter(marks__gt=80) # Greater than 80

Student.objects.filter(marks__gte=80) # At least 80

Student.objects.filter(marks__lt=40) # Less than 40

Student.objects.filter(marks__lte=40) # At most 40

Student.objects.filter(name__icontains="ali")

Limiting query results

Student.objects.all()[:5]

Returns a QuerySet containing up to five records.

Student.objects.order_by('-marks')[:3]

Returns the three highest-scoring students.

Student.objects.all()[5:10]

Skips five records and retrieves the next five.

Use `order_by()` when predictable ordering is important.

---

4. Django Views

a) Understanding Django views

A view receives an HTTP request and returns an HTTP response.

A view can retrieve database data, validate submitted information, apply business logic and render a template.

b) Function-Based Views (FBVs)

An FBV is a Python function that handles a request.

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Welcome Home")
```

Connecting it to a URL:

```
path('', views.home, name='home')
```

c) Class-Based Views (CBVs)

A CBV uses a Python class to organize request handling.

```
from django.views import View
from django.http import HttpResponse


class HomeView(View):
    def get(self, request):
        return HttpResponse("Welcome Home")
```

Connect the class to a URL:

```
path('home/', HomeView.as_view(), name='home')
```

`as_view()` converts the class into a callable view for Django's URL routing.

GET and POST in CBVs

```
class StudentView(View):
```

```
    def get(self, request):
        return HttpResponse("Display students")

    def post(self, request):
        return HttpResponse("Process submitted data")
```

- `get()`: Handles GET requests.
- `post()`: Handles POST requests.
- `self`: Refers to the current class instance.
- `request`: Contains details about the incoming request.

FBV vs CBV

FBV	CBV
Uses a function	Uses a class
Straightforward for simple logic	Organizes logic into methods
URL references the function	URL generally uses `.as_view()`
Request handling is written in the function	Methods such as `get()` and `post()` handle different HTTP methods

d) Returning HTML responses and templates

Plain HTTP response:

```
from django.http import HttpResponse

def home(request):
```

```
    return HttpResponse("<h1>Welcome</h1>")
```

Rendering a template:

```
from django.shortcuts import render


def home(request):
    return render(request, 'home.html')
```

`HttpResponse` returns a response directly. `render()` renders a template with optional context data and returns an HTTP response.

Context

Context is data passed from a view to a template, usually as a dictionary.

```
def student_details(request):
    context = {
        'name': 'Rashik',
        'marks': 90
    }

    return render(request, 'student.html', context)
```

In `student.html`:

```
<h1>{{ name }}</h1>
<p>Marks: {{ marks }}</p>
```

---

5. Django Templates

a) Introduction to Django Template Language (DTL)

DTL allows you to display dynamic data and control how HTML is rendered.

It commonly uses:

- Variables: `{{ variable }}`
- Tags: `{% tag %}`
- Filters: `{{ variable|filter }}`

b) Creating and using templates

Create the directory:

students/

└── templates/

    └── students/

        └── home.html

Example template:

<h1>Student Management</h1>

Welcome, {{ name }}

View:

```
def home(request):
    return render(
        request,
        'students/home.html',
        {'name': 'Rashik'}
    )
```

Django's template configuration must include the appropriate app-template settings. The standard project configuration normally enables `APP_DIRS`.

c) Template inheritance and extending templates

Template inheritance lets multiple pages share a common layout instead of repeating the same HTML.

base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
    <header>
        <h1>Student Portal</h1>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>
</body>
</html>
```

home.html

```
{% extends 'base.html' %}

{% block title %}
Home Page
{% endblock %}
```

{% block content %}

<h2>Welcome to the Student Portal</h2>

Manage student information here.

{% endblock %}

Important tags:

- `{% extends 'base.html' %}`: Inherits another template.
- `{% block title %}`: Defines a section that a child template can override.
- `{% block content %}`: Defines the page's main content section.
- `{% endblock %}`: Closes a block.

Place `{% extends %}` at or near the beginning of the child template, before its content blocks.

d) Template filters and tags

Filters modify the display of a value.

{{ name|upper }}

{{ name|lower }}

{{ name|title }}

{{ description|truncatewords:5 }}

{{ value|default:"Not provided" }}

{{ birthday|date:"d-m-Y" }}

Examples:

Filter	Purpose
`upper`	Converts text to uppercase
`lower`	Converts text to lowercase

`title`	Capitalizes words
`default`	Supplies a fallback for an empty or false value
`length`	Returns the length of a value
`truncatewords:5`	Limits text to five words
`date:"d-m-Y"`	Formats a date

Tags provide logic and template structure.

Conditional tag:

```
{% if marks >= 40 %}
   <p>Passed</p>
{% else %}
   <p>Failed</p>
{% endif %}
```

Loop tag:

```
{% for student in students %}
   <p>{{ student.name }}</p>
{% empty %}
   <p>No students found.</p>
{% endfor %}
```

URL tag:

View Students

CSRF tag:

{% csrf_token %}

Comment tag:

{% comment %}

This section is not rendered.

{% endcomment %}

---

6. URL Routing

a) Defining URL patterns

A URL pattern connects a URL path to a view.

from django.urls import path

from . import views

urlpatterns = [

    path('', views.home, name='home'),

    path('about/', views.about, name='about'),

]

b) Connecting views to URLs

A project can include an app's URL configuration.

Project `urls.py`:

from django.urls import path, include

urlpatterns = [

    path('students/', include('students.urls')),

]

App `students/urls.py`:

from django.urls import path

from . import views

app_name = 'students'

urlpatterns = [

    path('', views.student_list, name='list'),

    path('about/', views.about, name='about'),

]

The resulting URLs are:

- `/students/`
- `/students/about/`

`path()` defines a pattern. `include()` connects another URL configuration.

c) Namespacing URLs

An app can define:

app_name = 'students'

Then a URL can be referenced by its namespace and name:

{% url 'students:list' %}

Namespacing avoids ambiguity when different apps have identical URL names.

d) URL reversing

URL reversing finds a URL from its name rather than hardcoding its path.

In a template:

About

In Python:

from django.urls import reverse

url = reverse('students:about')

Redirecting to a named URL:

from django.shortcuts import redirect

return redirect('students:list')

If a URL pattern contains parameters, reversing must provide the required arguments or keyword arguments.

---

7. Practical Application — Student Management System

This mini-project combines models, migrations, views, templates and URL routing.

Step 1: Create the app

python manage.py startapp students

Add `'students'` to `INSTALLED_APPS` in `settings.py`.

Step 2: Define the model

In `students/models.py`:

from django.db import models

class Student(models.Model):

```
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()
    marks = models.IntegerField()

    def __str__(self):
        return self.name
```

Step 3: Create and apply migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 4: Create a view

In `students/views.py`:

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.order_by('name')

    context = {
        'students': students
    }

    return render(
        request,
        'students/student_list.html',
```

```
        context
    )
```

This retrieves students ordered by name and passes the QuerySet to the template.

Step 5: Create the template

Create `students/templates/students/student_list.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student List</title>
</head>
<body>
    <h1>Student List</h1>

    {% for student in students %}
        <div>
            <h2>{{ student.name }}</h2>
            <p>Age: {{ student.age }}</p>
            <p>Marks: {{ student.marks }}</p>
        </div>
    {% empty %}
        <p>No students available.</p>
    {% endfor %}
</body>
</html>
```

Step 6: Configure app URLs

Create `students/urls.py`:

```
from django.urls import path

from . import views


app_name = 'students'


urlpatterns = [

    path('', views.student_list, name='list'),

]
```

Step 7: Include app URLs in the project

In the project's `urls.py`:

```
from django.contrib import admin

from django.urls import path, include


urlpatterns = [

    path('admin/', admin.site.urls),

    path('students/', include('students.urls')),

]
```

Step 8: Run the server

```
python manage.py runserver
```

Visit:

http://127.0.0.1:8000/students/

The page displays the students stored in the database. If the database contains no students, it displays the empty-state message.

Step 9: Add students through the Django shell

python manage.py shell

Then:

```
from students.models import Student


Student.objects.create(
    name='Ali',
    age=18,
    marks=85
)


Student.objects.create(
    name='Aisha',
    age=19,
    marks=92
)
```

Refresh the browser to see the new records.

Optional extension: Add a student form

Create a simple HTML form:

```
<form method="POST">
    {% csrf_token %}

    <input type="text" name="name" placeholder="Name" required>
```

```
    <input type="number" name="age" placeholder="Age" required>
    <input type="number" name="marks" placeholder="Marks" required>

    <button type="submit">Add Student</button>
</form>
```

The view must handle both GET and POST requests, validate submitted values, save a `Student` object and redirect after successful submission. In a real application, use Django Forms or ModelForms for proper validation and error handling.

---

8. Additional Topics Covered

A. CSRF Protection

CSRF stands for Cross-Site Request Forgery. Django checks applicable unsafe requests using CSRF protection.

Add this inside an internal POST form:

{% csrf_token %}

Without a valid token, Django normally rejects the request with HTTP 403 when CSRF checks apply.

B. Middleware

Middleware processes requests and responses around the view.

Common middleware:

- `SecurityMiddleware`
- `SessionMiddleware`
- `CommonMiddleware`
- `CsrfViewMiddleware`
- `AuthenticationMiddleware`
- `MessageMiddleware`
- `XFrameOptionsMiddleware`

The order in `MIDDLEWARE` matters.

C. WSGI vs ASGI

- **WSGI:** Web Server Gateway Interface; designed around synchronous Python applications.
- **ASGI:** Asynchronous Server Gateway Interface; supports synchronous and asynchronous applications and protocols such as WebSockets.

Django generates `wsgi.py` and `asgi.py` as application entry points.

D. Synchronous vs asynchronous execution

Synchronous code waits for an operation to finish in the current execution flow.

Asynchronous code can yield control while waiting for non-blocking I/O, allowing other tasks to progress.

import asyncio

async def example():

    await asyncio.sleep(2)

    return "Done"

Async improves concurrency for suitable workloads; it does not automatically make every operation faster.

E. Debugging and configuration

- `DEBUG`: Enables detailed development error pages.
- `ALLOWED_HOSTS`: Defines accepted hostnames.
- `INSTALLED_APPS`: Lists enabled applications.
- `BASE_DIR`: Commonly identifies the project's base directory.
- `__str__()`: Must return a string.
- `request.POST`: Accesses submitted POST fields; it does not itself validate CSRF.

---

9. Final Revision Checklist

- Introduction, features and benefits of Django
- Project and app structure
- Installation and virtual environments
- Creating projects and apps
- `settings.py`, `urls.py`, `wsgi.py`, `asgi.py`
- Development server and management commands
- MVT architecture
- Models, fields and model methods
- Migrations and Django ORM
- CRUD operations and QuerySets
- Filtering, ordering, limiting and aggregation
- FBVs and CBVs
- `get()`, `post()` and `.as_view()`
- Templates and context
- Template inheritance and `{% extends %}`
- Template filters and tags
- URL patterns, `include()` and `path()`
- Namespaces and URL reversing
- GET and POST forms
- CSRF protection
- Middleware
- WSGI vs ASGI
- Synchronous vs asynchronous execution
- Practical Student Management System

Core concept: Django connects URL routing, views, models and templates to process requests and return responses. The ORM manages database interactions, context passes data to templates, and middleware provides processing and security around requests and responses.